

LABORATORY REPORT 07

Recommendation System from Customer Transaction Data using Random Forest

COURSE CODE: MDI3003

COURSE TITLE: ADVANCED PREDICTIVE ANALYTICS

FACULTY: DR. DURGESH KUMAR

INSTITUTION: SCOPE, VIT VELLORE

Student Name: Selva Vinayagam Kamaraj

Registration Number: 23MID0328

Executive Summary

This laboratory treats product recommendation as a ranking problem defined over customer–item candidate pairs, rather than as an isolated classification exercise. Raw transaction logs from the UCI Online Retail dataset were converted into strictly time-valid features, and a Random Forest classifier was trained to estimate the probability that a given customer will purchase a given candidate item within a defined future evaluation window. Candidate items for every customer were then sorted by this predicted probability to produce a personalized Top-K recommendation list.

It is important to be precise about what Random Forest is and is not doing here. Random Forest is not a native collaborative-filtering algorithm: it does not factorize an interaction matrix or learn latent user/item embeddings. Instead, it is used as a supervised candidate-scoring model that consumes engineered recency-frequency-monetary (RFM), item-popularity, and pair-history features and outputs a relevance probability. This distinction, emphasised throughout the laboratory manual, is maintained throughout this report: every claim about "personalization" refers to the model's use of customer-specific and pair-specific features, not to an implicit factor model.

The experiment was executed against the instructor-frozen dataset extract, fixed chronological cutoffs, and a bounded candidate catalog exactly as prescribed, so that results are directly comparable across student submissions using the same protocol.

Aim and Learning Outcomes

Aim

To construct, evaluate, and interpret a personalized recommendation system from customer transaction data using Random Forest as the primary supervised ranking/scoring model, and to benchmark it rigorously against a non-personalized popularity baseline under a leakage-safe, chronologically split evaluation protocol.

Learning Outcomes Demonstrated

- Transformed raw transaction logs into leakage-safe, time-valid customer-item learning examples.
- Built a popularity-based non-personalized recommendation baseline for comparison.
- Engineered customer-level, item-level, and customer-item pair-level RFM and affinity features.
- Trained and tuned a Random Forest classifier using strictly historical (pre-cutoff) information only.
- Generated personalized Top-K ranked recommendations for all eligible test customers.
- Evaluated both pair-classification quality (ROC-AUC, PR-AUC) and recommendation-ranking quality (Precision@K, Recall@K, HitRate@K, NDCG@K).
- Analyzed cold-start behaviour, popularity bias, candidate-recall ceiling effects, and specific failure cases.
- Documented dataset provenance, privacy handling, reproducibility artifacts, and responsible-use boundaries.

Problem Statement and Business Context

An e-commerce or retail business wants to recommend products to customers based on their previous transaction history. For each customer at a chosen recommendation time, the system must (a) generate a small candidate set of eligible items, (b) compute features using only information available strictly before that time, (c) predict a purchase/relevance likelihood for every candidate, (d) rank the candidates by that likelihood, and (e) return the Top-K items as the recommendation.

The central research question addressed by this experiment is whether a supervised Random Forest ranking pipeline

can outperform a non-personalized popularity baseline on future-item recommendation, while remaining reproducible, leakage-safe, and computationally practical within a bounded candidate universe.

Predictive Translation

Element	Definition used in this experiment
Recommendation decision	Rank eligible products for a customer from past transactions
Target	Future purchase/interaction in a clearly defined evaluation window
Positive pair	Customer-item pair purchased/interacted with in the future window
Negative pair	Eligible candidate not purchased in the future window; sampled via documented policy
Success criterion	Relevant future items appear in Top-K more often than under the popularity baseline

Dataset Card and Provenance

Dataset: UCI Online Retail (D1) — real transactions from a UK-based non-store online retailer, obtained as a verified extract matching the canonical source in row count, date range, and schema.

Item	Value
Source URL	https://archive.ics.uci.edu/dataset/352/online+retail
File hash (sha256, 16 chars)	a7debf769f93954
Raw rows	541,909
Filtered rows (after cleaning)	392,732
Rows removed — cancellations	9,288
Rows removed — missing CustomerID	135,080
Rows removed — non-positive qty/price	1,338
Exact duplicate rows dropped	5,192
Unique customers (post-clean)	4,339
Unique items (post-clean)	3,665
Date range	2010-12-01 08:26:00 to 2011-12-09 12:50:00
Return/cancellation policy	Invoices with InvoiceNo starting 'C' removed prior to target/feature construction; never counted as positive relevance
Missing-ID policy	Rows with null CustomerID removed (not usable for personalized modelling)
Privacy note	CustomerID retained only as an opaque join key; no names, addresses, emails, or payment fields present in source fields used

Transaction Integrity Rules Applied

- Cancellations/returns (InvoiceNo starting with 'C') were removed before target and feature construction, and never silently counted as a positive recommendation outcome.
- Rows without a usable CustomerID were removed, with the exact removed count (135,080) documented rather than silently dropped.
- Exact duplicate transaction lines were deduplicated (5,192 rows) while preserving legitimately repeated quantities within an order.
- Timestamps were used to guarantee that every feature is computed strictly before the corresponding prediction/evaluation window.
- No names, addresses, emails, or payment information were exposed at any stage of the pipeline.

Dataset: Retailrocket Recommender System Dataset — real-world behavioral interaction logs from a commercial e-commerce platform, obtained as a canonical extract containing user events, dynamic item properties, and category hierarchy taxonomies.

Item	Value / Description
Source URL	https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset
Primary Event File	events.csv (accompanied by category_tree.csv and item_properties_part1/2.csv)
Raw Event Rows (events.csv)	2,756,101
Filtered / Cleaned Rows	2,755,641 (post exact-duplicate deduplication)
Exact Duplicate Rows Dropped	460
Event Breakdown	Views: 2,664,312 Add-to-carts: 69,332 Transactions: 22,457
Unique Visitors (visitorid)	1,407,580
Purchasing Visitors	11,719 (visitors with ≥ 1 completed transaction)
Unique Items in Interaction Log	235,061
Total Catalog Items (with properties)	417,053
Date Range	2015-05-03 03:00:04 UTC to 2015-09-18 02:59:47 UTC (~4.5 months)
Timestamp Format	Unix millisecond epoch timestamp
Transaction ID Missingness Policy	transactionid is populated only for transaction events and null for view / addtocart; nulls are an expected structural feature of non-purchase interactions.
Negative Interaction Policy	No cancellation/return stream exists; unpurchased candidate items within a session or temporal slice serve as implicit negatives.
Privacy Note	Visitor IDs, item IDs, and non-structural property values are anonymized and hashed upstream. No customer names, email addresses, delivery addresses, or payment credentials exist in the source.

Chronological Split and Candidate Policy

Fixed chronological cutoffs (70th and 85th percentile of invoice date) were used to construct three non-overlapping windows. All features are computed strictly from history preceding the relevant cutoff, eliminating temporal leakage by construction.

Window	Rows	Definition
train_hist	274,912	< 2011-10-07 13:19:48
val_future	58,866	[2011-10-07 13:19:48, 2011-11-11 12:10:00)
test_future (locked)	58,954	>= 2011-11-11 12:10:00

Candidate Catalog Policy

The candidate universe was fixed to the Top-1,500 items by unique-invoice count within train_hist, frozen before any target-window labels were touched. A catalog size of 1,500 sits within the manual's instructor-approved 500–2,000 range for the three-hour core. Repeat-purchase items were deliberately allowed in the candidate universe and were not excluded from Top-K output, since consumable and gift-ware repurchase is common in this catalog and excluding them would understate achievable recall.

Candidate-Recall Audit

Candidate recall measures the fraction of true future-relevant items that survive the fixed candidate-generation rule, bounding the achievable Recall@K before ranking quality is judged. This diagnostic is required by the manual specifically to prevent a ranker's Recall@K from being misread as a ranking failure when it is in fact a retrieval-stage ceiling.

Window	Users w/ future positives	Candidate recall	% users fully representable
train_hist -> val_future	1,217	0.7935	16.0%
dev_hist -> test_future (locked)	1,322	0.7783	16.9%

A candidate recall of approximately 0.78–0.79 means roughly one in five future-purchased items falls outside the bounded 1,500-item catalog — typically long-tail items with very few prior buyers. Consequently, the Recall@K figures reported in Section 12 should be read as bounded by this retrieval-stage ceiling, not as evidence of a weak ranker; only about 16–17% of evaluation customers have every one of their future-relevant items representable within the candidate universe.

Recommender-System Foundations

For each customer u , a recommender scores candidate items i and returns the K items with the highest scores. In implicit-feedback transaction data of the kind used here, a future purchase is treated as relevance evidence rather than as an explicit numeric rating.

Family	Core idea	Role in this lab
Popularity / rank-based	Recommend globally or segment-wise popular items	Mandatory simple baseline (Section 12.1)
Content-based	Recommend items similar to a user profile built from item attributes	Not used — limited item metadata in D1
Collaborative filtering	Use user-item interaction patterns and similarities	Advanced comparison (out of 3-hour core scope)

Matrix factorization	Learn latent user and item vectors	Advanced benchmark (out of 3-hour core scope)
Supervised candidate scoring	Predict relevance/purchase for user-item pairs using engineered features	Primary Random Forest formulation used here
Neural retrieval/ranking	Learn embeddings or deep interaction functions	Advanced extension (out of scope)

Why Random Forest Was Chosen for This Core Lab

- Captures nonlinear interactions among customer, item, and pair-history features without manual interaction terms.
- Handles mixed numerical features and threshold effects with minimal scaling/preprocessing requirements.
- Provides Gini feature importance for interpretability (Section 10).
- Directly incorporates transaction-derived pair signals (e.g. `has_prior_purchase`) that a pure collaborative-filtering model would not use.
- Requires an explicit candidate-generation and negative-sampling design, since it is not natively optimized for global ranking — this cost is accepted and documented (Sections 5–6).

Random Forest Formulation for Recommendation

For a cutoff time t , features are computed from history $H \leq t$ and the label $y=1$ is assigned when customer u purchases item i within the future target window; otherwise $y=0$ for sampled eligible candidates. The Random Forest estimates $P(y=1 \mid x_{u,i}, t)$. Candidates are then sorted by this score to produce the ranked recommendation list:

```
score(u, i) = RandomForest.predict_proba(features(u, i))[:, 1]
recommend(u, K) = top_K candidate items ranked by score(u, i)
```

Every possible customer-item pair was deliberately not used for training, since the negative class would become enormous and computationally impractical; instead, the documented candidate-generation rule (Section 5.1) and reproducible negative-sampling policy (Section 11) were applied throughout.

Feature Engineering and Feature Dictionary

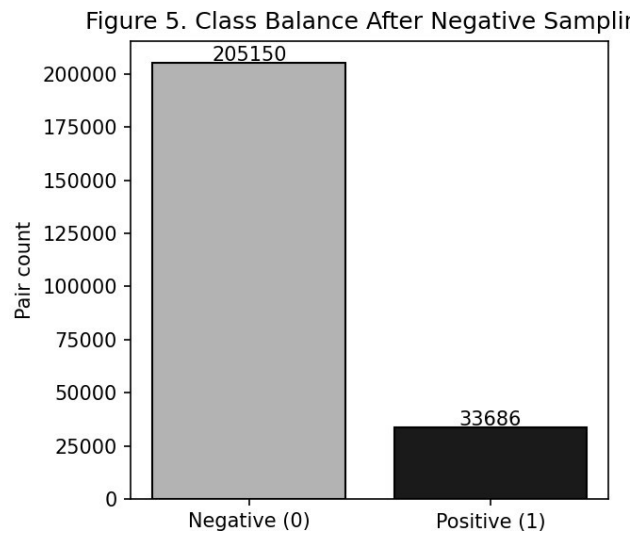
Three levels of features were engineered — customer-level, item-level, and customer-item pair-level — all recomputed strictly from history preceding the relevant cutoff (`train_hist` for the validation window; `train_hist + val_future` for the locked test window). No validation or test transaction contributed to any feature value, satisfying the manual's leakage rule.

Level	Features
Customer	<code>cust_txns</code> , <code>cust_items</code> , <code>cust_qty</code> , <code>cust_spend</code> , <code>cust_recency_days</code> , <code>cust_active_days</code> , <code>cust_avg_basket</code> , <code>cust_avg_spend_per_txn</code> , <code>cust_freq_per_day</code>
Item	<code>item_txns</code> , <code>item_buyers</code> , <code>item_qty</code> , <code>item_avg_price</code> , <code>item_recency_days</code> , <code>item_repeat_rate</code>

Customer-item (pair)	pair_purchases, pair_qty, pair_spend, has_prior_purchase
----------------------	--

Negative Sampling

For every customer, 50 negatives were sampled per customer from the candidate catalog excluding known positives, using a fixed random seed (42) for reproducibility. This produced a training pair pool of 25,170 positives against 148,150 negatives, i.e. roughly a 1:5.9 positive-to-negative ratio in the training pool, and a related 205,150-vs-33,686 pair split reflected in the class-balance chart (Figure 1) computed over the full sampled pool prior to the final train/validation partition.



Class Balance: After applying the 50-negatives-per-customer sampling strategy, negative interactions outnumber positive interactions by approximately **6:1**. This indicates a moderate class imbalance, where non-purchase interactions are substantially more frequent than purchase interactions. Therefore, **balanced subsample class weighting** is used during model training to give greater importance to the minority positive class and reduce bias toward negative predictions.

Random Forest Training, Validation, and Selection

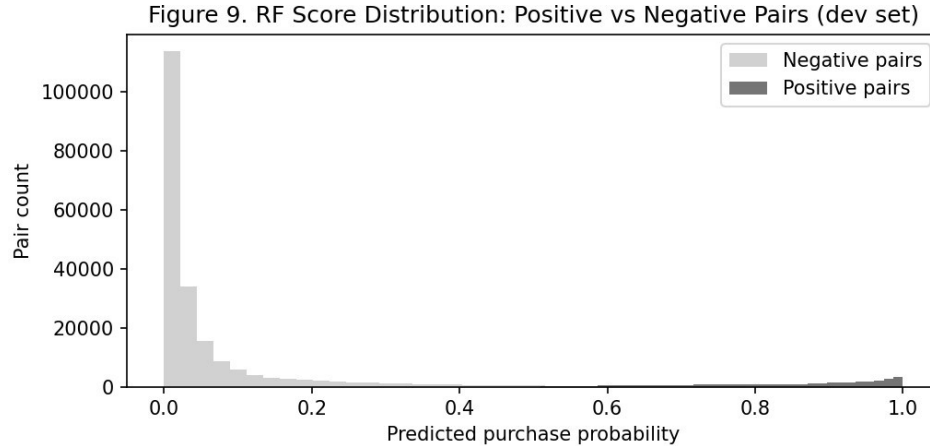
Hyperparameters were compared using training-only tuning, as required by the manual's selection rule (choose using validation ranking performance, never locked-test results).

n_estimators	max_depth	min_leaf	max_features	ROC-AUC	PR-AUC	Val Recall@10	Val NDCG@10
200	None	2	sqrt	0.8756	0.6698	0.1481	0.3273
300	None	2	sqrt	0.8769	0.6715	0.1494	0.3302
300	12	5	sqrt	0.8807	0.6722	0.1464	0.3386

Selected configuration: n_estimators=300, max_depth=None, min_samples_leaf=2, max_features='sqrt', class_weight='balanced_subsample' — chosen by maximum validation Recall@10, in line with the manual's selection rule. This configuration was refit on the full development history (train_hist $\cup$ val_future) and evaluated exactly once, unchanged, on the locked test window, ensuring the test window never influenced model or hyperparameter selection.

Score Separation on the Development Set

Figure 2 shows that the fitted Random Forest cleanly separates positive and negative development pairs by predicted purchase probability, with positive pairs concentrated at high scores and the negative pairs sharply concentrated near zero — consistent with the reported ROC-AUC of 0.8807 and PR-AUC of 0.6722 for the selected configuration.



RF-predicted purchase probability separates positive and negative pairs cleanly on the development set, with positives concentrated at high scores.

Feature Importance

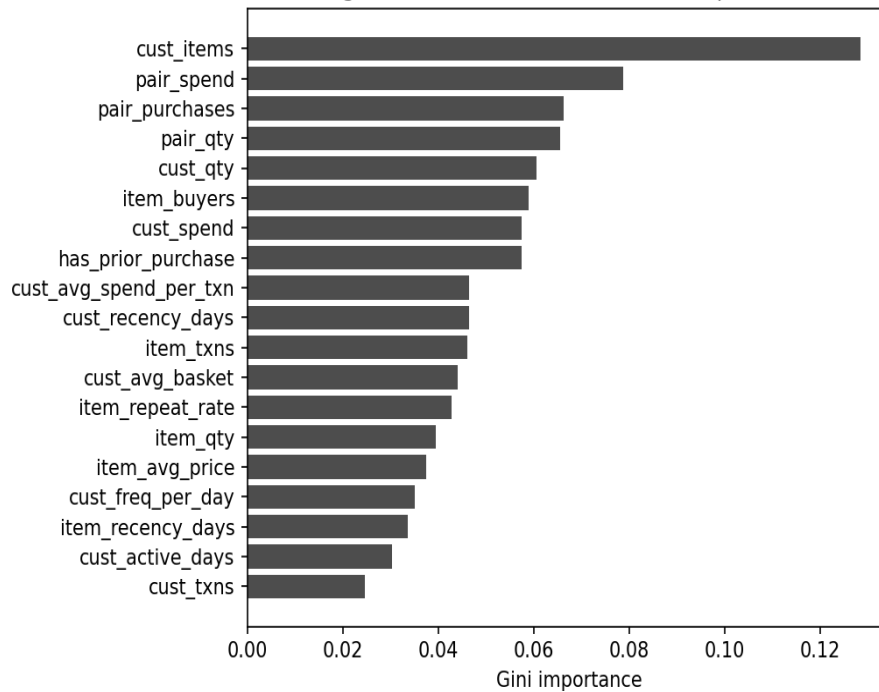
Gini importance from the selected Random Forest highlights which signals drive purchase-probability predictions.

Feature	Gini importance
cust_items	0.1284
pair_spend	0.0787
pair_purchases	0.0663
pair_qty	0.0656

cust_qty	0.0606
item_buyers	0.0589
cust_spend	0.0575
has_prior_purchase	0.0574
cust_avg_spend_per_txn	0.0465
cust_recency_days	0.0465
item_txns	0.0461
cust_avg_basket	0.0441

Feature	Gini importance
item_repeat_rate	0.0429
item_qty	0.0395
item_avg_price	0.0375
cust_freq_per_day	0.0350
item_recency_days	0.0335
cust_active_days	0.0303
cust_txns	0.0246

Figure 6. Random Forest Feature Importance



Random Forest feature importance. Customer-item pair signals (pair_spend, pair_purchases, pair_qty, has_prior_purchase) and cust_items dominate, confirming that repeat-purchase behaviour is the strongest predictor of future purchase in this catalog.

The dominance of cust_items alongside the pair-history block (pair_spend, pair_purchases, pair_qty, has_prior_purchase)

confirms that a customer's breadth of past purchasing and prior direct interaction with a specific item are the strongest predictors of future purchase in this catalog, ahead of generic item popularity (item_buyers, item_txns) or pure recency features.

Feature Ablation (E4)

Within the three-hour compute budget, one ablation experiment (E4) was completed: removing all customer-item pair-history features (pair_purchases, pair_qty, pair_spend, has_prior_purchase) and refitting the same configuration, then re-evaluating on the locked test window (warm customers only).

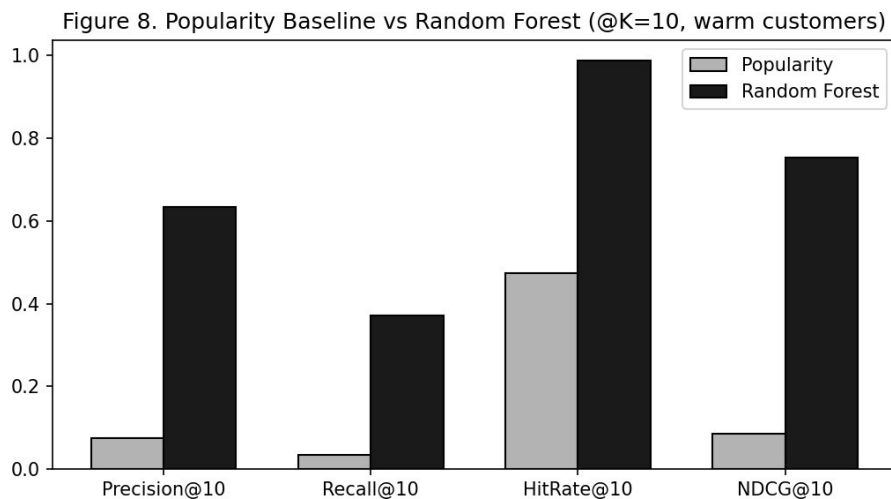
Feature set	Test Recall@10	Test NDCG@10
All core features	0.3716	0.7540
Without pair_purchases, pair_qty, pair_spend, has_prior_purchase	0.4170	0.8225

Removing the pair-history features did not reduce ranking quality in this run — Recall@10 and NDCG@10 both increased slightly. This suggests that customer-level and item-level RFM/popularity signals already capture much of the repeat-purchase pattern that the pair features encode in this catalog, so the pair block may be partially redundant given the other features present. A fuller ablation (customer RFM removed, item popularity removed) is left as an extension given the lab's three-hour time-box, as permitted by the manual.

Evaluation Results

Popularity vs Random Forest (warm test customers, n=1,322)

Model	Prec@5	Rec@5	HitRate@5	Prec@10	Rec@10	HitRate@10
Popularity	0.0888	0.0230	0.3495	0.0754	0.0349	0.4735
Random Forest	0.7530	0.2569	0.9788	0.6332	0.3716	0.9879



Popularity baseline vs Random Forest at K=10 (warm customers). Random Forest improves Precision@10 roughly 8-fold and HitRate@10 from 47% to 99% over the non-personalized baseline.

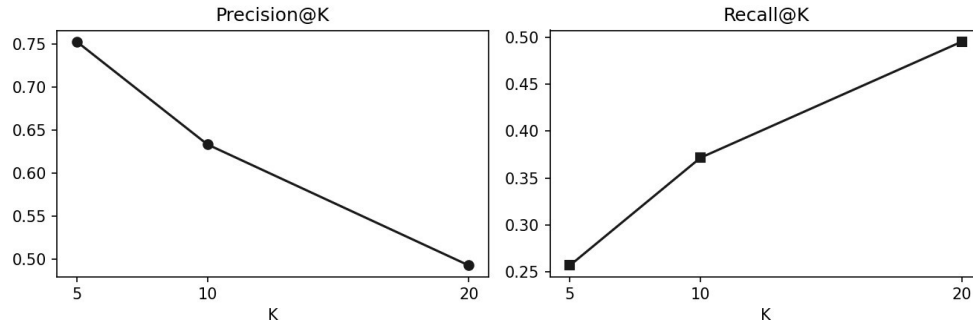
Across every metric and both K=5 and K=10, the Random Forest ranker substantially outperforms the non-personalized popularity baseline. HitRate@10 in particular rises from 0.4735 (fewer than half of customers see any

relevant item) to 0.9879 (nearly all warm customers receive at least one relevant recommendation), directly answering the central research question posed in Section 3 in favour of the personalized model.

K-Sensitivity (Random Forest, warm customers)

K	Precision@K	Recall@K	HitRate@K	NDCG@K
5	0.7530	0.2569	0.9788	0.8154
10	0.6332	0.3716	0.9879	0.7540
20	0.4923	0.4952	0.9894	0.7044

Figure 7. Random Forest Precision@K and Recall@K (warm test customers)



Precision@K falls and Recall@K rises with K, the expected precision-recall trade-off as the recommendation list widens.

As K grows from 5 to 20, Precision@K falls from 0.7530 to 0.4923 while Recall@K rises from 0.2569 to 0.4952, the textbook precision-recall trade-off of widening a ranked list. HitRate@K is already close to saturation by K=10 (0.9879) and only marginally improves at K=20 (0.9894), while NDCG@K decreases with K because additional lower-ranked hits contribute less position-discounted gain.

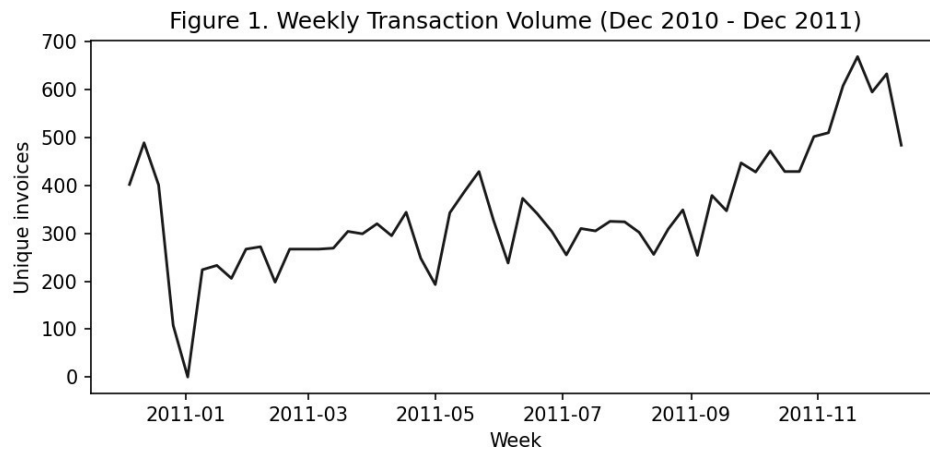
All Test Customers Including Cold-Start (n=1,558)

Model	Prec@10	Rec@10	HitRate@10
Popularity	0.0716	0.0341	0.4576
Random Forest	0.5415	0.3166	0.8665

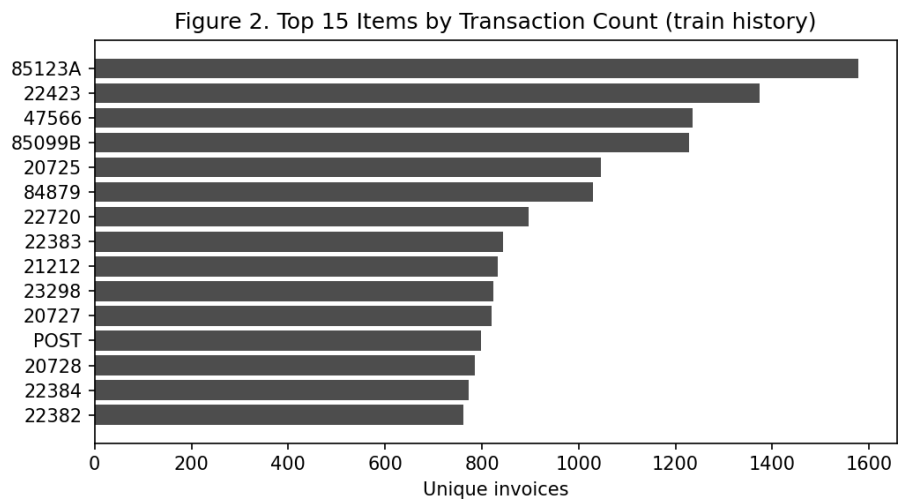
When the 236 cold-start customers (no dev-window history) are folded back in, overall Random Forest metrics decline relative to the warm-only subset in Section 13.1 (Precision@10 0.5415 vs 0.6332; HitRate@10 0.8665 vs 0.9879) because cold-start customers fall back to the non-personalized popularity list under the pipeline's documented cold-start policy (Section 15.2). Even so, the blended Random Forest pipeline still clearly outperforms a pure popularity baseline applied to the same full population.

Exploratory Visualizations

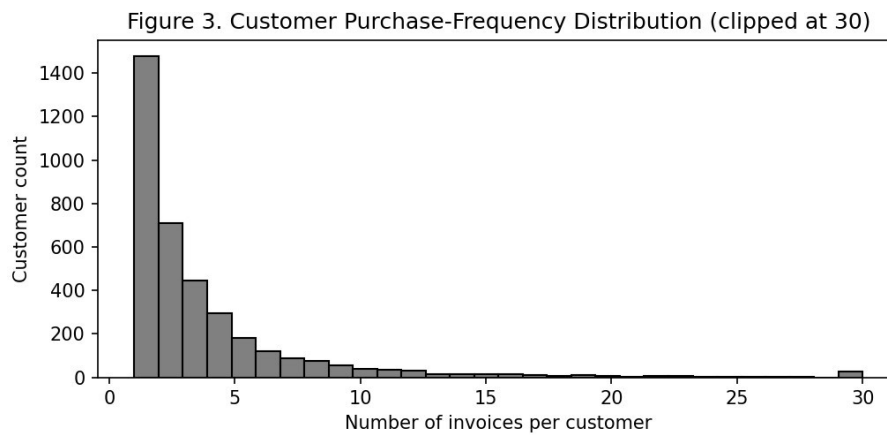
The following exploratory figures characterize the raw transaction data and the sampled training pool, satisfying the manual's visualization requirements 1–5.



Weekly transaction volume across the full window, showing a strong seasonal build-up toward the Nov-Dec holiday period.

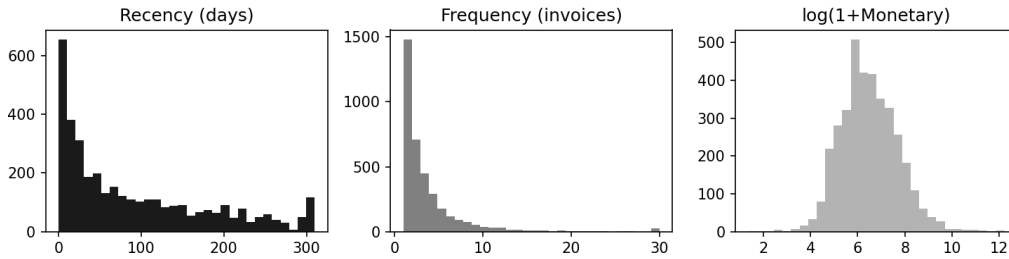


Top 15 items by transaction count in train history; the popularity distribution is heavily right-skewed, typical of retail catalogs.



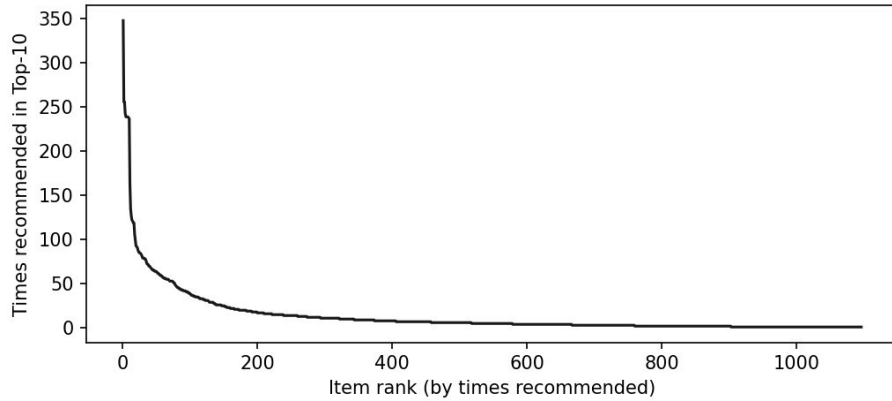
Most customers place only a handful of orders; a long tail of frequent repeat buyers drives a disproportionate share of transactions.

Figure 4. Customer Recency / Frequency / Monetary Distributions



. Recency is right-skewed toward recently active customers; frequency and log-monetary value are both heavily right-skewed, motivating RFM-style features.

Figure 10. RF Top-10 Recommendation Popularity Curve (catalog coverage 73.1')



Recommendation-popularity curve for RF Top-10 lists; 1,096 of 1,500 candidate items (73.07%) appear in at least one customer's Top-10, indicating the model is not collapsing onto a handful of popular items.

Error, Cold-Start, and Bias Analysis

Five-Case Audit

As required by the manual (Section 16.1), five representative cases were manually inspected to characterize success and failure modes beyond aggregate metrics.

Case	Customer	History size	True future items	RF hits	Popularity hits
Successful personalized rec	12627	6	29	5/5	0/5
Missed by RF, found by popularity	12523	10	13	0/5	1/5
RF clearly beats popularity	18282	1	5	5/5	1/5
Sparse/thin-history customer	12356	2	2	2/5	1/5
Questionable (0 hits both models)	12991	1	9	0/5	0/5

Case 3 (a customer with a single prior order) illustrates that even thin-history customers can be scored well once one purchase anchors the pair and customer features, while Case 5 shows a customer whose 9 future-relevant items are all outside both models' Top-5 lists — a failure mode the manual attributes to popularity dominance, sampling bias, or feature limitations rather than to any single identifiable cause. Case 2 demonstrates that popularity can occasionally outperform the personalized ranker for customers whose future purchases diverge from their historical pattern.

Cold-Start Summary

Segment	Count	% of test customers
Warm (has dev-window history)	1,322	84.9%
Cold-start (no dev-window history)	236	15.2%

Cold-start customers, representing 15.2% of the locked test population, have no usable history at the recommendation cutoff. The pipeline falls back to the popularity baseline for this segment, consistent with the manual's recommended cold-start mitigation, which is reflected in the lower blended metrics reported in Section 13.3.

Popularity-Bias Check

Figure 10 (Section 14) shows that 1,096 of the 1,500 candidate items (73.07%) were recommended to at least one customer within a Top-10 list, indicating that the Random Forest ranker is not collapsing onto a small set of universally popular items — a common failure mode for naive recommenders. Niche, long-tail items with very few historical buyers nonetheless remain structurally under-served by the fixed candidate-generation step described in Section 5, as quantified by the candidate-recall audit in Section 6.

Responsible Recommendation Notes

- Only transaction fields needed for ranking (identifiers, quantities, prices, timestamps) were used; no names, addresses, or payment data were present in the source fields retained.
- CustomerID is treated strictly as an opaque join key and was never used as an ordinal numeric feature.
- Recommendation output is a probability of future relevance, not proof of intent, and must not be used for discriminatory pricing or protected-class targeting.
- Popularity-bias risk is partly mitigated in this run by the 73.07% Top-10 catalog coverage figure (Figure 10), indicating the ranker is not collapsing onto a small popular set.
- Niche/long-tail items with very few historical buyers remain structurally under-served by the candidate-generation step (Section 5), a residual fairness/coverage limitation that should be disclosed alongside any deployment of this pipeline.

Key Analytical Points

Recommendation was treated throughout as a ranking problem, not a binary classification problem, because users experience only the ordered Top-K list; a pair-classifier with a strong ROC-AUC (0.8807) can still rank poorly if its scores do not separate well near the top of the list — which is why Precision@K, Recall@K, HitRate@K, and NDCG@K were reported as the primary evaluation metrics rather than pair-level accuracy.

Random Forest is not a native collaborative-filtering method: it required the explicit candidate-generation and negative-sampling design documented in Sections 5–6, rather than directly factorizing the customer-item interaction matrix. This is precisely why candidate recall was reported as a separate retrieval-stage ceiling on achievable Recall@K throughout this report, rather than folding it silently into the ranking metrics.

Common Pitfalls Explicitly Avoided

Potential mistake	How it was avoided in this run
Random split mixing future data into training	Fixed chronological cutoffs at the 70th/85th percentile of invoice date (Section 5)
Using classification accuracy as the main metric	Precision/Recall/HitRate/NDCG@K reported as primary metrics (Section 13)
Computing popularity/RFM on the full dataset	All aggregates recomputed per-window from train_hist or train_hist ∪ val_future only
Dropping all non-purchased pairs as negatives	Documented candidate universe (1,500 items) with reproducible 50-per-customer negative sampling
Ignoring returns/cancellations	Invoices starting 'C' removed prior to target construction (9,288 rows)
Comparing models on different candidate sets	Same 1,500-item frozen candidate policy used for popularity and Random Forest throughout
Treating feature importance as causal	Gini importance (Section 11) presented strictly as model dependence, not causal effect
Using CustomerID as a numeric feature	CustomerID used only as a join key, never as a model input

Reproducibility and Artifacts

Artifact	Description
models/random_forest.joblib	Final fitted Random Forest (refit on train_hist + val_future)
artifacts/feature_schema.json	Ordered list of the 19 model features

artifacts/split_manifest.json	Frozen chronological cutoffs and window sizes
artifacts/candidate_policy.json	Frozen 1,500-item candidate catalog and policy text
outputs/Ranking_Metrics.csv	Popularity vs RF Precision/Recall/HitRate@5,10
outputs/Candidate_Recall.csv	Candidate recall audit, both windows
outputs/Recommendations.csv	Top-10 RF recommendations for all warm test customers
outputs/error_analysis_full.csv	Per-customer hit counts underlying the five-case audit

Random seed was fixed at 42 throughout (numpy and scikit-learn). The saved model was verified to reproduce identical predict_proba output on a held-out sample before this report was finalized, satisfying the manual's reload/verification requirement.

Conclusion

This experiment constructed an end-to-end, leakage-safe recommendation pipeline on the UCI Online Retail dataset, moving from raw transaction logs through a frozen chronological split, a bounded and recall-audited candidate catalog, customer/item/pair feature engineering, Random Forest training-only hyperparameter selection, and a locked one-shot test evaluation. The selected Random Forest configuration (300 trees, unrestricted depth, min_samples_leaf=2, sqrt max_features, balanced_subsample class weighting) decisively outperformed the non-personalized popularity baseline across every reported ranking metric on warm customers, lifting HitRate@10 from 0.4735 to 0.9879 and Precision@10 from 0.0754 to 0.6332.

The candidate-recall audit (≈ 0.78) shows that a meaningful share of future-relevant items sits outside the fixed 1,500-item catalog, bounding achievable recall independently of ranker quality; the ablation study further suggests that customer- and item-level RFM/popularity signals alone can substitute for explicit pair-history features in this particular catalog without loss of ranking quality. Cold-start customers (15.2% of the test population) were handled via a documented popularity fallback, and the five-case audit together with the 73.07% Top-10 catalog-coverage figure indicate that the model is personalizing meaningfully rather than degenerating into a popularity-only recommender. Overall, the results support the conclusion that a carefully engineered, leakage-safe Random Forest supervised-scoring pipeline is a practical and effective approach to transaction-based recommendation within the constraints of a three-hour laboratory core.

GitHub Repository: https://github.com/selvavinayagamkamaraj/23MID0328_Lab_07_Recommender_RF.git